

IT EL 5 – ADVANCED INFORMATION MANAGEMENT

Development of an Integrated Materials and Inventory Ticketing System
for Rivera Sarvida Construction Inc.

by

Josef Kristian T. Asuncion

Kim Ralfonzo D.F. Diwa

John Andrew M. Manota

Renz Lindonne J. Rulona

BSIT 3-6

1. Project Context

Project Background

The RSCI Operations System moves a construction material request from the site engineer to stock in the warehouse inside one MySQL database. Rivera-Sarvida Construction Inc. (RSCI) currently tracks this with QuickBooks and paper. The prototype is a scaled-down rebuild of RSCI's live procurement ticketing website, which runs on Firebase. It uses MySQL, a plain HTML/JavaScript front end and a small Express API. No Firebase is involved.

The prototype leaves out the billing and variation-order cascade, subcontractors, notifications, photo attachments and Excel export. The case study focuses on what a relational database must guarantee for a procurement workflow: integrity, atomic updates, protected data, limited access and fast reports.

Objectives

- Give every request one traceable pipeline from filing to stock.
- Prevent double approvals and double payments.
- Protect sensitive supplier and payment data (supplier TIN, check numbers, staff phone numbers, passwords).
- Limit each staff role to what its job needs, enforced by the database and not only by the application.
- Keep expense reports fast on a large ledger (seeded with 100,000 rows).

Users

Role	What the role does	Database account
Engineer	Files material requests (tickets); edits and resubmits a rejected ticket	rsci_engineer
Boss	Approves or rejects tickets and purchase orders	rsci_boss
Office Manager (OM)	Approves or rejects purchase orders; read-only elsewhere	rsci_om
P.O. Officer	Creates purchase orders and vendors	rsci_po_officer
Accountant	Records the payment and posts it to expenses	rsci_accountant
Inventory	Receives delivered goods; stocks items in and out	rsci_inventory
Administrator	Read-only oversight of every action; cannot act or see secrets	rsci_admin

A separate `rsci_auth` account exists only for the login and registration pages.

New accounts are created on the website. The registration form asks for a name, email, optional phone, role and password. The list of roles comes from the `roles` table in `sql/01_schema.sql`, so adding, renaming or hiding a role is a change to that SQL file (`self_register = 0` hides a role from the form), followed by `npm run setup-db`.

Business Processes

- 1 The engineer files a request. It starts as *Pending Boss Approval*.
- 2 The Boss approves it (*Approved for PO*) or rejects it with a reason. The submitter can edit and resubmit a rejected ticket.
- 3 The P.O. Officer creates a purchase order with a unique P.O. number. The ticket becomes *PO Created*.
- 4 The Boss or the OM approves the P.O.; either sign-off alone is enough.
- 5 The Accountant records the payment (check number encrypted) and posts it to Expenses. The P.O. becomes *Purchased*.
- 6 Inventory receives the goods into stock. The P.O. becomes *Received* and the ticket becomes *Completed*.

Database Requirements and Design

The database `rsci_sql` is in third normal form on InnoDB with utf8mb4. Requirements and how the schema meets them:

- Relational integrity: foreign keys between all linked tables, `UNIQUE` P.O. numbers, and `CHECK` constraints on quantities, prices and stock.
- Atomic multi-table updates: InnoDB transactions (Section 2).
- Encrypted sensitive fields: `VARBINARY` columns `tin_enc`, `check_no_enc` and `phone_enc` (Section 3).
- Per-role privileges: one MySQL account per role with column-level grants (Section 4).
- Fast reports on a large ledger: a covering index on `expenses` (Section 5).

Table	Purpose	Key relationships
<code>roles</code>	Role catalogue: code, label, description, whether it can be chosen on the registration form	Referenced by <code>users.role</code>
<code>users</code>	Staff accounts; bcrypt password hash, encrypted phone	<code>roles</code> ; referenced by tickets, P.O.s, payments, logs
<code>projects</code>	Company and project names, unique together	Referenced by tickets, expenses, inventory log
<code>vendors</code>	Suppliers; encrypted TIN	Referenced by purchase orders
<code>tickets</code>	Material requests and their status	<code>projects</code> , <code>users</code> (requester, decider)
<code>ticket_items</code>	Line items of a request	<code>tickets</code> (cascade delete)
<code>purchase_orders</code>	P.O. with unique <code>po_no</code> , status and total	<code>tickets</code> , <code>vendors</code> , <code>users</code>
<code>po_items</code>	P.O. lines with quantity and unit price	<code>purchase_orders</code> (cascade delete)
<code>payments</code>	One payment per P.O.; encrypted check number	<code>purchase_orders</code> (unique), <code>users</code>
<code>expenses</code>	General Expenses ledger (100,000 seeded rows)	<code>projects</code> , <code>purchase_orders</code> , <code>users</code>
<code>inventory</code>	Stock on hand per item and unit	Referenced by <code>inventory_log</code>

Table	Purpose	Key relationships
inventory_log	Every stock movement	inventory, purchase_orders, projects, users
activity_log	Append-only audit trail	users

The schema is in `sql/01_schema.sql`.

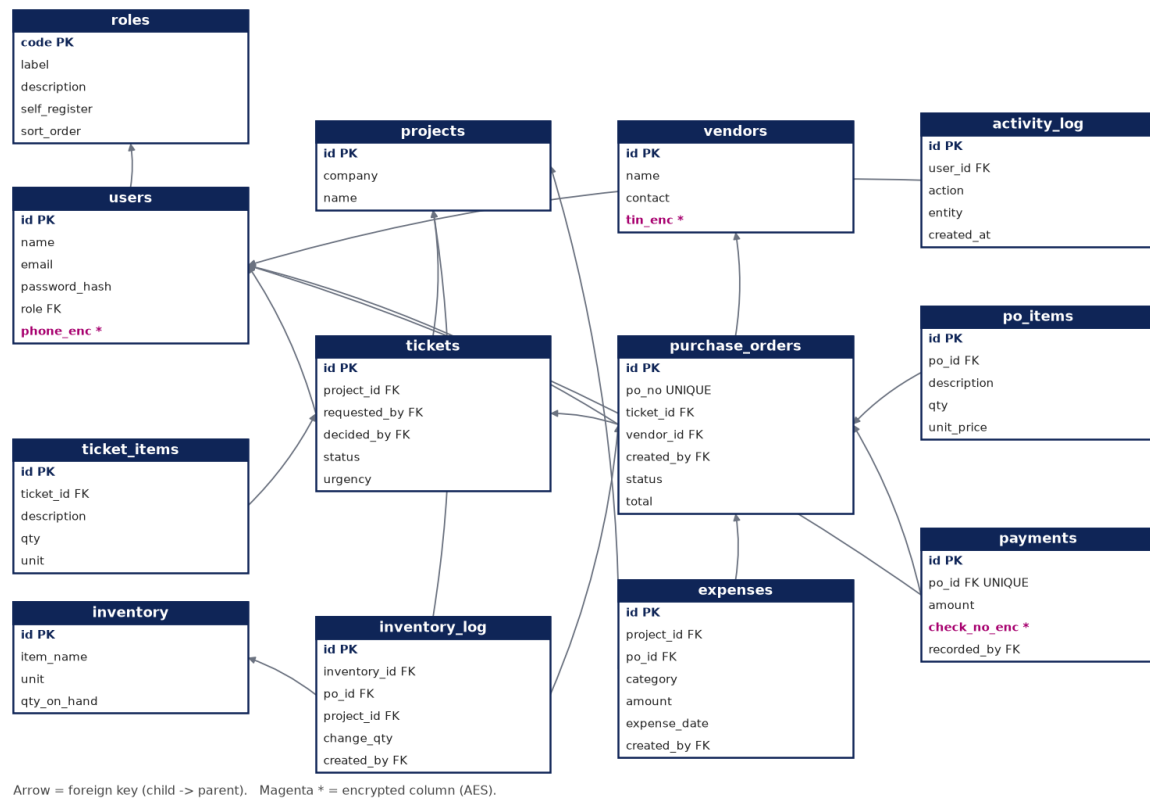


Figure 1. Entity relationship diagram of the `rsci_sql` database.

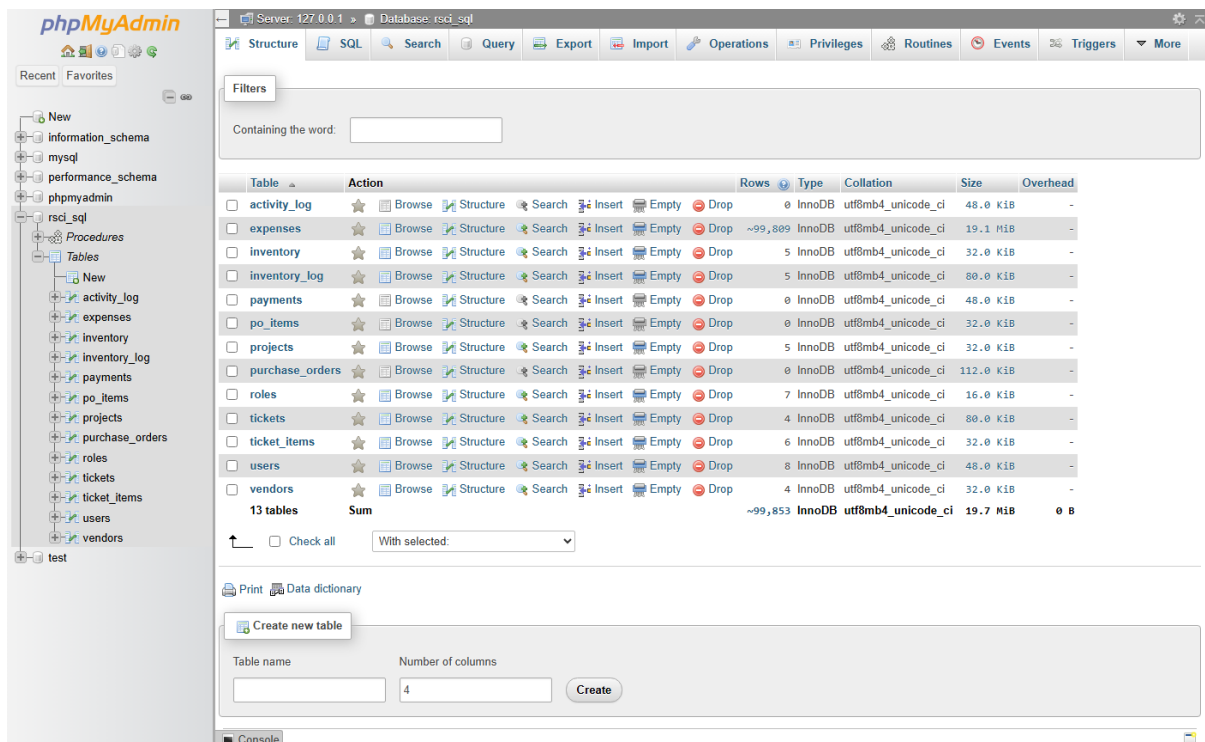


Figure 2. The 13 tables of rsci_sql in phpMyAdmin (InnoDB, utf8mb4).

2. Transaction Management

Every step that changes more than one table runs inside one InnoDB transaction, so it either completes entirely (COMMIT) or leaves no trace (ROLLBACK). The helper withTransaction in server/db.js opens the transaction, runs the steps, commits, and rolls back on any error.

```
START TRANSACTION;
UPDATE tickets SET status = 'PO Created' WHERE id = 3 AND status = 'Approved for PO';
INSERT INTO purchase_orders (po_no, ticket_id, vendor_id, created_by, total)
VALUES ('PO-DEMO-001', 3, 1, 4, 1500.00);
SET @po := LAST_INSERT_ID();
INSERT INTO po_items (po_id, description, qty, unit, unit_price)
VALUES (@po, 'Cement 40kg', 10, 'bag', 150.00);
COMMIT;
```

Transactions in the System

Business action	Steps that succeed or fail together	Where
Create a P.O.	Flip the ticket to <i>PO Created</i> ; insert the P.O.; insert its items; write the activity log	server/routes/purchaseOrders.js
Confirm purchase (Accountant)	Move the P.O. to <i>Purchased</i> ; insert the payment with encrypted check number; post to expenses; write the log	server/routes/purchaseOrders.js
Receive goods (Inventory)	Add items to stock; write the stock movement log; close the P.O. and its ticket	server/routes/purchaseOrders.js
Stock out	Reduce stock only if enough is on hand; write the movement log	sql/03_transactions_demo.sql
Edit and resubmit a rejected ticket	Check the owner and status; replace the items; reset the status	resubmit_ticket procedure

ACID Properties

- **Atomicity.** A duplicate P.O. number raises error 1062 on the insert. The earlier ticket status change is rolled back, so ticket 4 stays *Approved for PO*. The interface also has a *Simulate failure before COMMIT* switch that throws an error after every statement succeeded, to show that ROLLBACK undoes all of them.
- **Consistency.** Foreign keys, `UNIQUE` P.O. numbers and `CHECK` constraints (quantity above zero, stock not below zero) reject any state that breaks the rules.
- **Isolation.** Status changes are guarded updates such as `UPDATE ... SET status = 'Approved' WHERE id = 1 AND status = 'Pending Approval'`. The first session takes the row lock. A second approver waits, then sees 0 rows affected, and the API returns HTTP 409. A P.O. therefore cannot be approved twice or paid twice.
- **Durability.** InnoDB writes committed changes to its redo log, so they survive a server restart.

Savepoints, Isolation Levels and Error Handling in SQL

- **SAVEPOINT.** A transaction can undo part of its work: `ROLLBACK TO SAVEPOINT after_item_a` removes the second insert and keeps the first, and `COMMIT` saves only what remains.
- **Isolation levels.** InnoDB uses `REPEATABLE READ`: a transaction keeps seeing the same stock figure even after another session commits a change. With `READ COMMITTED` the second read would already show the new value. The two-session script is in `sql/03_transactions_demo.sql`.
- **Error handling.** The stored procedure `sp_create_po` runs the whole purchase-order transaction inside MySQL. `DECLARE EXIT HANDLER FOR SQLEXCEPTION` rolls back and `RESIGNAL` re-raises the original error, so the caller still sees it. `SIGNAL` refuses a ticket that is not *Approved for PO* with a clear message.

```
CALL sp_create_po('PO-DEMO-002', 4, 1, 4, 85.50); -- commits: ticket, P.O., items and log together
CALL sp_create_po('PO-DEMO-002', 3, 1, 4, 85.50); -- ERROR 1062: handler rolls back, ticket 3 unchanged
```

Stock Cannot Go Below Zero

The guard sits in the `WHERE` clause. Releasing 30 bags of cement affects 1 row and is committed. Releasing 99,999 bags affects 0 rows, is rolled back, and writes no log row.

```
UPDATE inventory SET qty_on_hand = qty_on_hand - 99999
WHERE id = 1 AND qty_on_hand >= 99999; -- 0 rows affected
```

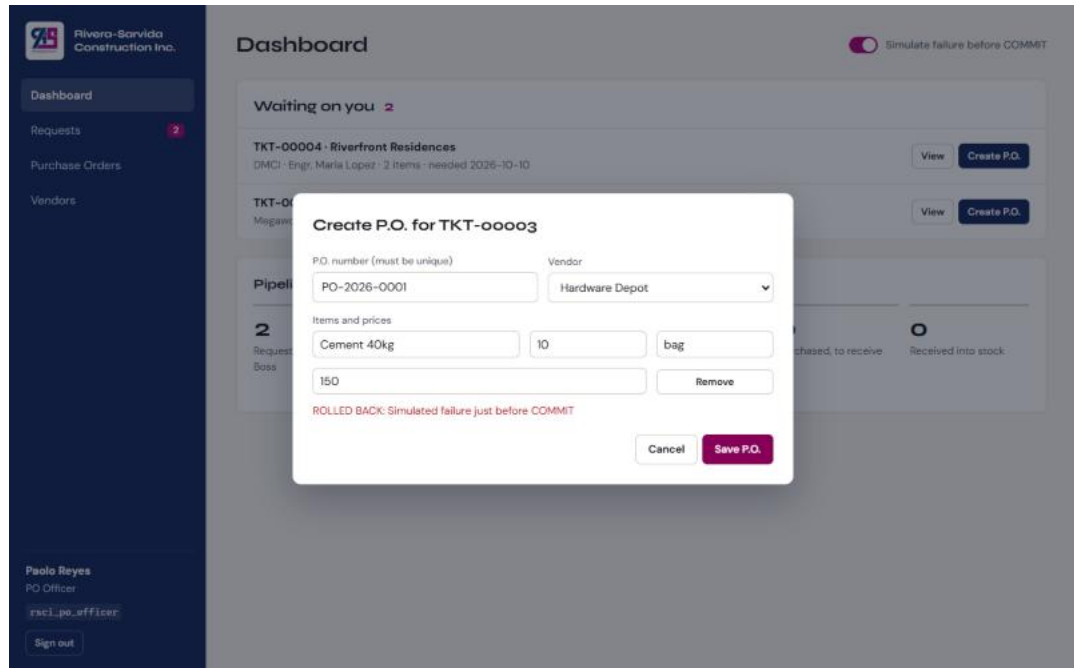


Figure 3. Creating a P.O. with Simulate failure before COMMIT on: the app reports ROLLED BACK, and ticket 3 stays Approved for PO with no P.O. row saved.

3. Database Encryption

Four kinds of sensitive data are protected at rest, with the method chosen to fit how each is used.

Data	Column	Method	Reversible
Supplier tax ID (TIN)	vendors.tin_enc	AES encryption	Yes, with the key
Check / reference number	payments.check_no_enc	AES encryption	Yes, with the key
Staff phone number	users.phone_enc	AES encryption	Yes, with the key
Login password	users.password_hash	bcrypt hash	No

How It Works

The encrypted columns are `VARBINARY(255)`, so the database stores raw ciphertext. The cipher is MySQL's default AES (128-bit, ECB mode). The application encrypts on write and decrypts on read with MySQL's own functions. The key is `SHA2(ENC_KEY, 256)`.

```
-- write (server/db.js encrypt)
INSERT INTO payments (po_id, amount, check_no_enc, recorded_by)
VALUES (?, ?, AES_ENCRYPT('CHK-0001', SHA2('<ENC_KEY>', 256)), ?);
```

```
-- read (server/db.js decrypt)  
SELECT CAST(AES_DECRYPT(check_no_enc, SHA2('<ENC_KEY>', 256)) AS CHAR) FROM payments;
```

- The key is not in the database. ENC_KEY lives in the server's .env file. Someone who copies the database or a backup sees only ciphertext.
- Passwords are hashed, not encrypted. bcrypt is one-way, so nobody can recover a password, including the administrator.
- Access control backs this up. Only the roles that need a secret can read its column (Section 4). The Engineer, OM and Inventory accounts cannot even select tin_enc.

Demonstration

sql/06_encryption_demo.sql shows four results side by side:

- 1 The stored hex value next to the decrypted value for vendor TIN, check numbers and phone numbers.
- 2 A query with the wrong key, which returns NULL.
- 3 The users table showing only bcrypt hashes.
- 4 The phone column stored as hex.

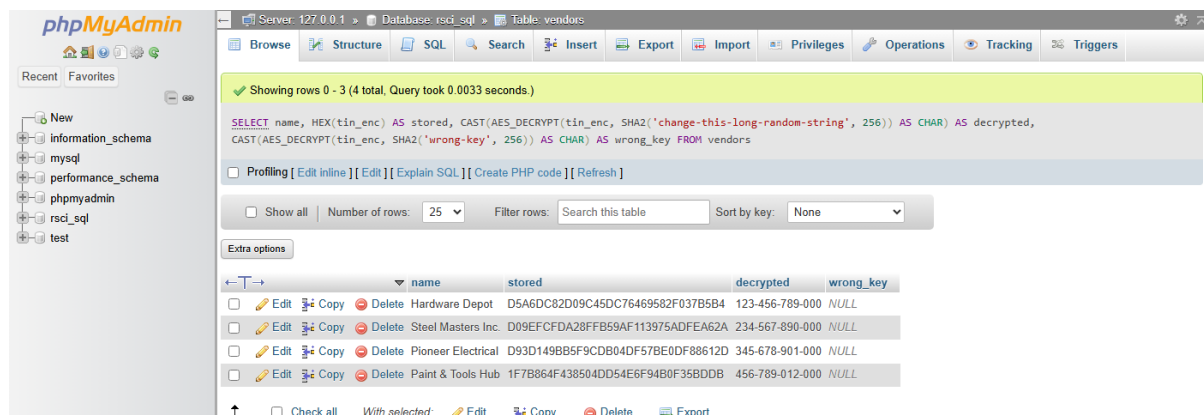


Figure 4. Vendor TIN stored as ciphertext (stored), decrypted with the right key, and returning NULL with a wrong key.

4. Database Authorization and Privileges

MySQL itself enforces who can do what. The API opens a connection as a different MySQL user for each application role (poolFor in server/db.js), so a bug in the application cannot give a role more access than its grants allow. All accounts and grants are in sql/02_roles.sql.

Privilege Matrix

Role	Can write	Can read	Cannot see
rsci_auth	Insert a user (name, email, password_hash, role, phone_enc only) and an audit-log row	users login columns, including password_hash; roles	Everything else
rsci_engineer	Insert tickets and items	Projects, inventory, vendor name and contact	tin_enc, payments, expenses
rsci_boss	Insert projects, tickets; update ticket status, decided_by, reject_reason; update P.O. status, decided_by	All financial tables, including payments	Nothing hidden from the owner
rsci_om	Update P.O. status, decided_by	P.O.s, expenses, inventory, activity log	tin_enc, payments
rsci_po_officer	Insert vendors, P.O.s and items; update ticket status	Projects, inventory, vendors	Payments, expenses
rsci_accountant	Insert payments and expenses; update P.O. status	P.O.s, vendors, payments, expenses	Approval columns
rsci_inventory	Insert and update stock; insert stock log; update P.O. and ticket status	P.O.s, vendor name and contact	tin_enc, payments
rsci_admin	Nothing	Every table for oversight	password_hash, phone_enc, tin_enc, check_no_enc

Principles Applied

- Least privilege. Each account gets only the statements and tables its job needs.
- Column-level grants. `GRANT SELECT (id, name, contact) ON vendors` hides the encrypted TIN from roles that do not need it. `GRANT UPDATE (status, decided_by) ON purchase_orders` lets an approver change two columns and nothing else.
- No `DELETE` for any role. Records are never removed. A submitter edits a rejected ticket only through the `resubmit_ticket` stored procedure, which runs as `SQL SECURITY DEFINER` and checks inside that the caller owns the ticket and that it is in *Rejected by Boss*. Any other caller gets error 1644.
- Append-only audit trail. Most roles have only `INSERT` on `activity_log`, so nobody can alter or erase their own actions.
- Separation of duties. The Boss approves but does not pay. The Accountant pays but does not approve. The Administrator sees everything but changes nothing.

Account Registration and Role Configuration

The registration page runs as `rsci_auth`. Its grant lists the only columns it may write, so a new account cannot set its own id or creation date: `GRANT INSERT (name, email, password_hash, role, phone_enc) ON users TO rsci_auth`. The role must exist in `roles` because `users.role` is a foreign key. The user row and its audit-log entry are saved in one transaction. After registering, the person is signed in through the MySQL account that matches the chosen role, so the same grants apply to new accounts as to the seeded ones.

Self-registration is for demonstration. A real system would hide the Boss and Administrator roles from the form (`self_register = 0`) and let an administrator assign them.

MySQL Roles

Besides one account per application role, `sql/07_roles_demo.sql` shows a real MySQL role object: privileges are granted once to a role, the role is granted to users, and a REVOKE on the role changes every member at once.

```
CREATE ROLE r_site_viewer;  
GRANT SELECT ON rsci_sql.projects TO r_site_viewer;  
GRANT SELECT ON rsci_sql.inventory TO r_site_viewer;  
GRANT r_site_viewer TO 'demo_viewer'@'localhost';  
REVOKE SELECT ON rsci_sql.inventory FROM r_site_viewer; -- every member loses it
```

REVOKE Demonstration

`sql/05_revoke_demo.sql` is run as root while the OM is logged in to the application:

```
SHOW GRANTS FOR 'rsci_om'@'%';  
REVOKE SELECT ON rsci_sql.activity_log FROM 'rsci_om'@'%';  
-- The OM's Activity Log page now fails: SELECT command denied  
GRANT SELECT ON rsci_sql.activity_log TO 'rsci_om'@'%';  
-- Access is restored
```

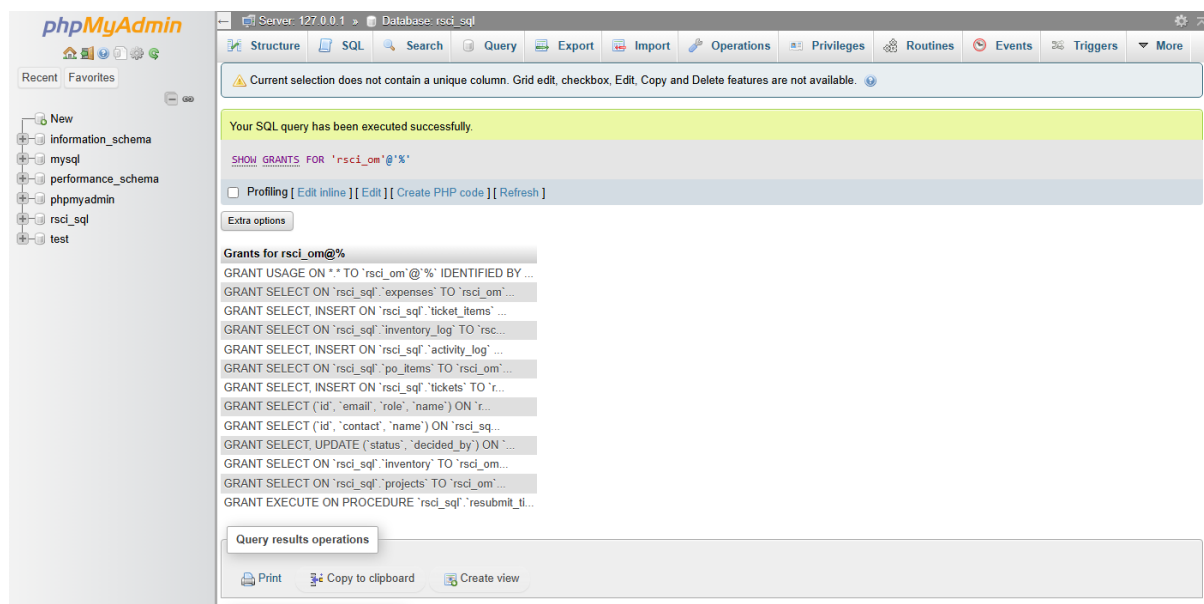


Figure 5. `SHOW GRANTS FOR rsci_om` in phpMyAdmin: `SELECT` only on most tables, column-level grants on users and vendors, `UPDATE` limited to `status` and `decided_by`.

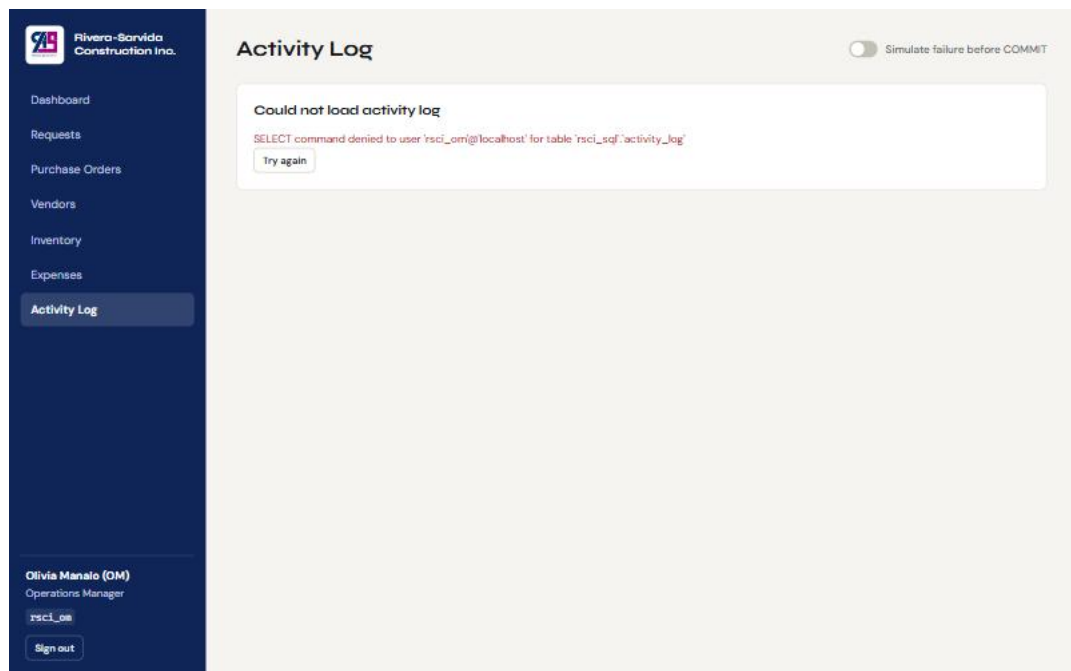


Figure 6. After REVOKE SELECT ON activity_log from rsci_om, the OM Activity Log page fails with "SELECT command denied". The grant was restored afterwards.

5. Query Optimization

The General Expenses report totals spending per project for a date range. It is used by the Boss and the Accountant, and it runs against the 100,000-row `expenses` table. A covering composite index lets MySQL answer it from the index alone. The script is

sql/04_optimization.sql.

```
SELECT project_id, COUNT(*) AS entries, SUM(amount) AS total
FROM expenses
WHERE expense_date BETWEEN '2026-03-01' AND '2026-03-31'
GROUP BY project_id;
```

Before and After

	Before the index	After the index
Index available	Only the foreign-key indexes; none on <code>expense_date</code>	<code>idx_expenses_date_proj_amt</code> (<code>expense_date</code> , <code>project_id</code> , <code>amount</code>)
type	index (scans the whole foreign-key index <code>fk_e_project</code>)	range
Extra	Using where	Using where; Using index (covering), plus Using temporary; Using filesort for the GROUP BY
Rows examined	99,809	4,791 (about 5%)

The figures above come from `EXPLAIN` run in phpMyAdmin on the seeded database (Figures 7 and 8).

```
CREATE INDEX idx_expenses_date_proj_amt ON expenses (expense_date, project_id, amount);
```

Why This Index

- Range column first. `expense_date` leads, so the `BETWEEN` filter becomes a range scan over only the matching entries.
- Covering. `project_id` (grouped) and `amount` (summed) are in the index, so MySQL never reads the table rows.
- One index serves the report. It replaces a scan of the whole ledger with a read of about one twentieth of it.

The Anti-Pattern

Wrapping the indexed column in a function defeats the index, because MySQL must compute the function for every row:

```
WHERE YEAR(expense_date) = 2026 AND MONTH(expense_date) = 3 -- not sargable
```

The script runs this form too, so the plan can be compared with the range form. The index can be dropped with `DROP INDEX idx_expenses_date_proj_amt ON expenses;` to repeat the demonstration.

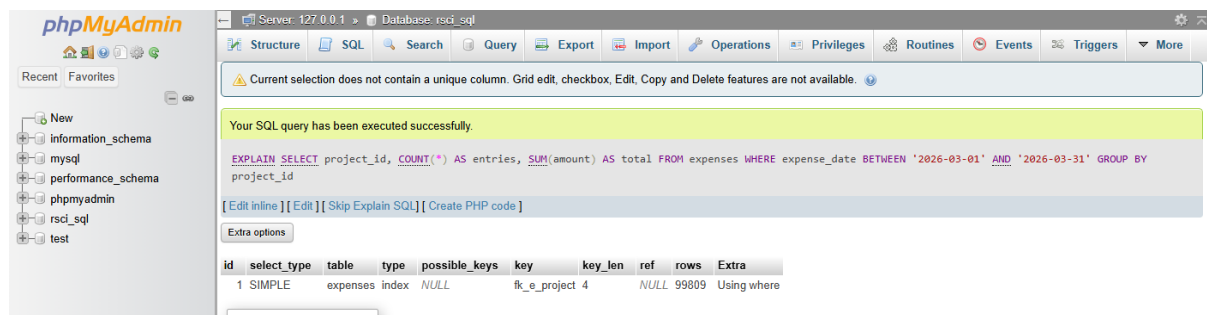


Figure 7. EXPLAIN before the index: type = index, 99,809 rows examined.

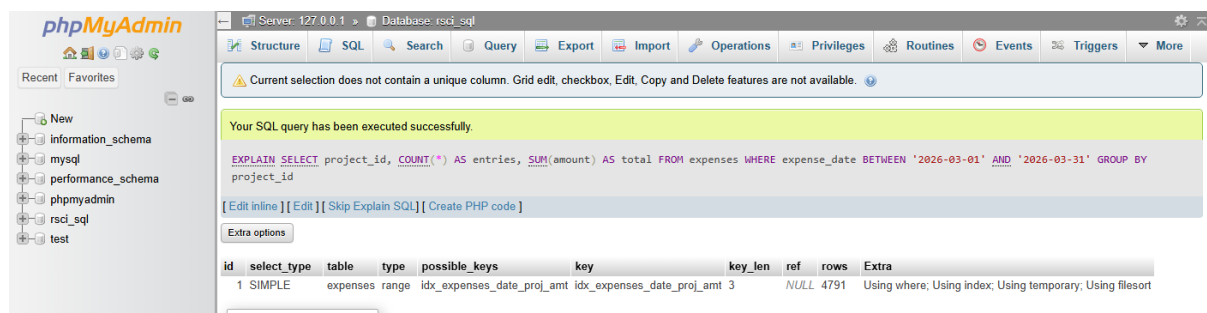
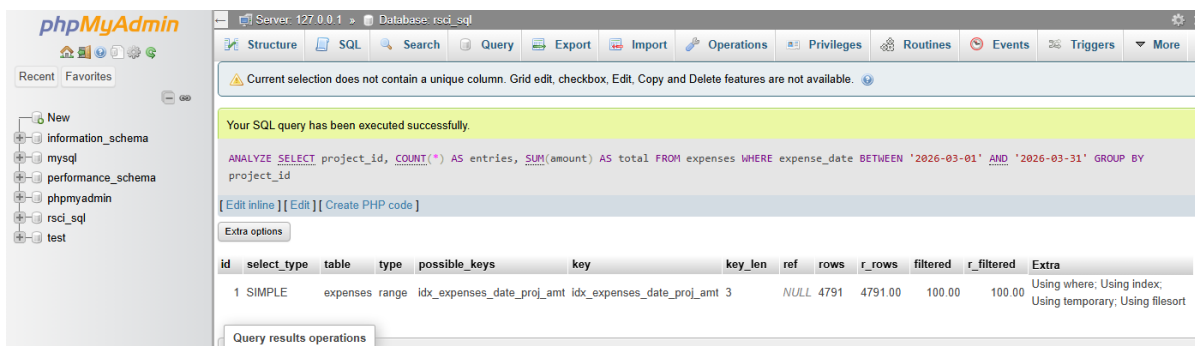


Figure 8. EXPLAIN after the index: type = range, key idx_expenses_date_proj_amt, 4,791 rows examined.

Measured Timings: ANALYZE

EXPLAIN only predicts. On MariaDB, `ANALYZE SELECT ...` runs the query and adds measured columns (`r_rows`, `r_filtered`). On MySQL 8.0.18 or later the same thing is `EXPLAIN ANALYZE SELECT ...`, kept as a comment in `sql/04_optimization.sql`. Here the estimate (4,791) and the measured rows (4,791) agree.



id	select_type	table	type	possible_keys	key	key_len	ref	rows	r_rows	filtered	r_filtered	Extra
1	SIMPLE	expenses	range	idx_expenses_date_proj_amt	idx_expenses_date_proj_amt	3	NULL	4791	4791.00	100.00	100.00	Using where; Using index; Using temporary; Using filesort

Figure 9. ANALYZE after the index: estimated rows and measured r_rows are both 4,791.

Query Rewriting

The same month can be asked two ways. WHERE YEAR(expense_date) = 2026 AND MONTH(expense_date) = 3 hides the column inside functions, so MySQL must compute them for every row. WHERE expense_date >= '2026-03-01' AND expense_date < '2026-04-01' compares the bare column and can use the index.

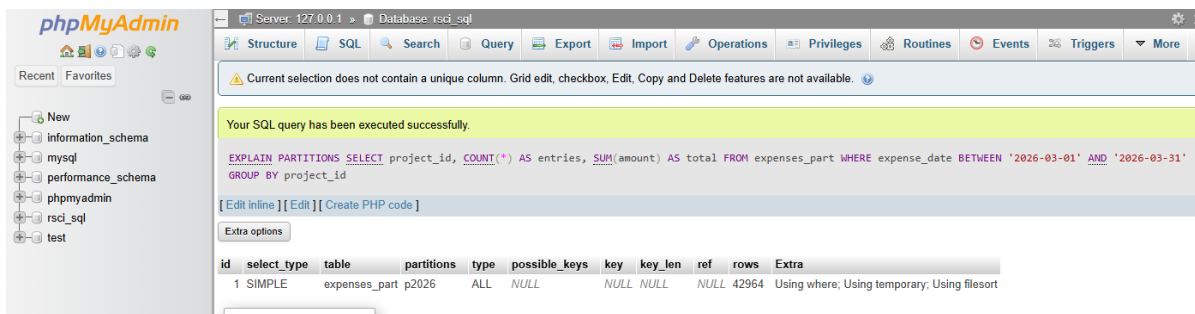
Efficient Joins

To report totals per project name, a correlated subquery runs once per project. The faster form aggregates the 100,000-row table once through the index and then joins the few summary rows to projects on its primary key. Joins are made on indexed columns, and only the needed columns are selected.

```
SELECT p.company, p.name, s.entries, s.total
FROM (SELECT project_id, COUNT(*) AS entries, SUM(amount) AS total
      FROM expenses
      WHERE expense_date BETWEEN '2026-03-01' AND '2026-03-31'
      GROUP BY project_id) AS s
JOIN projects p ON p.id = s.project_id;
```

Partitioning

A demo copy of the ledger, expenses_part, is partitioned by year with PARTITION BY RANGE (YEAR(expense_date)). A query for one month reads only the partition for that year (partition pruning). A partitioned InnoDB table cannot have foreign keys and the partition column must be in the primary key, so the live expenses table stays unpartitioned.



id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	expenses_part	p2026	ALL	NULL	NULL	NULL	NULL	4296.4	Using where; Using temporary; Using filesort

Figure 10. EXPLAIN PARTITIONS: only partition p2026 is read for a March 2026 query.